

Optimisation Methods to Address Variational Bayesian Posterior Variance Collapse

David Ewing (82171165)

3 May 2026

Abstract

Variational Inference (VI) recasts Bayesian posterior computation as an optimisation problem, making it a natural testbed for comparing the gradient-based methods taught in ENEL445. This document provides the mathematical derivations for VI applied to Bayesian linear regression under a mean-field Normal–Gamma approximation.

The Evidence Lower Bound (ELBO) is derived for a Normal–Gamma variational approximation, with Coordinate Ascent Variational Inference (CAVI) used as the analytical baseline. Gradient ascent, Newton’s method, and BFGS are then applied to the same variational objective. Cholesky and log-space reparameterisations are used to keep the covariance matrix symmetric positive definite and the Gamma parameters strictly positive during unconstrained optimisation.

Two numerical test cases are used. The first is a linear Bayesian regression problem, where the data-generating relationship is linear in the predictor. The second is a quadratic Bayesian regression problem, where the response depends on both x and x^2 . Although the second problem is nonlinear in the input variable, it remains linear in the coefficient vector after feature expansion, allowing both test cases to use the same Bayesian regression and VI framework.

The report also examines posterior variance collapse, a known failure mode of mean-field VI caused by the exclusive KL divergence and factorisation assumptions. Entropy regularisation is considered as a principled modification of the ELBO to encourage broader posterior approximations. The optimisation methods are compared using convergence behaviour, final ELBO value, posterior mean accuracy, posterior standard deviation, numerical stability, and agreement with a Gibbs-sampling reference.

Introduction

Bayesian inference provides principled uncertainty quantification but requires computing intractable posterior distributions. For most real-world models, the posterior integral,

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})}{\int p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}},$$

cannot be evaluated analytically. Variational Inference (VI) converts this inference problem into an optimisation problem, making it directly amenable to the gradient-based methods in this course.

Instead of computing the true posterior, VI finds the “best” approximation $q(\boldsymbol{\theta})$ from a tractable family $Q(\boldsymbol{\theta})$ by maximising the Evidence Lower Bound (ELBO). This transforms intractable integration into tractable optimisation—a natural testbed for the gradient-based methods.

A well-known failure mode of mean-field VI is *Posterior Variance Collapse*: the optimised approximation $q(\boldsymbol{\theta})$ is systematically overconfident, underestimating posterior uncertainty. This is a direct consequence of the KL direction minimised during ELBO maximisation, compounded by the mean-field factorisation assumption.¹

This paper addresses variance collapse via two complementary strategies,

- The first applies new optimisation methods to the *standard* ELBO objective: gradient ascent, Newton’s method, and BFGS are compared to determine whether the choice of optimiser affects dispersion quality.
- The second modifies the objective function itself, augmenting the ELBO with an entropy regularisation term $\lambda H[q(\boldsymbol{\beta})]$ to create a new objective $\mathcal{L}_{\text{reg}}(\boldsymbol{\phi}) = \mathcal{L}(\boldsymbol{\phi}) + \lambda H[q(\boldsymbol{\beta})]$ that explicitly incentivises broader posteriors.

Both the optimiser-comparison and modified-ELBO strategies are explored.

Throughout all gradient-based methods, the variational covariance $\boldsymbol{\Sigma}_{\boldsymbol{\beta}}$ must remain symmetric positive definite at every iteration.² Standard gradient steps in Euclidean space can violate this requirement, leading to numerical failure. To preserve positive definiteness during optimisation, the covariance matrix is parameterised using its Cholesky factor. This converts the constrained covariance optimisation problem into an unconstrained problem over real-valued parameters, allowing gradient ascent, Newton’s method, and BFGS to be applied safely.

¹See [Appendix B](#) for the full mechanistic explanation.

²This structural constraint is satisfied automatically via Cholesky reparameterisation; see [Appendix C](#).

Key Notation

Symbol	Meaning
β	Regression coefficient vector
$\mathbf{u}$	Random effects vector (hierarchical model)
ϵ	Observation noise vector
τ_e	Observation-noise precision
τ_u	Random-effects precision
μ_β	Mean of variational posterior $q(\beta)$
Σ_β	Covariance of variational posterior $q(\beta)$
$\mathbf{L}$	Lower-triangular Cholesky factor of Σ_β
η_a, η_b	Log-space parameters for a_e, b_e

Scope of the Numerical Study

This report addresses the optimisation-method requirements by formulating variational inference for Bayesian regression as a constrained optimisation problem and applying four optimisation approaches: Coordinate Ascent Variational Inference (CAVI), gradient ascent, Newton’s method, and BFGS. Each method is described in terms of its objective function, entry conditions, search direction, step-size rule, and exit conditions. This ensures that the numerical comparison is based not only on final accuracy, but also on the optimisation behaviour of each method. For all gradient-based line-search methods, the proposed search direction must be an ascent direction. That is, before a step is accepted, the direction $\mathbf{d}^{(t)}$ must satisfy

$$\nabla \text{ELBO}(\phi^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (1)$$

This condition ensures that the line search is well-posed for maximising the ELBO. Gradient ascent uses the gradient itself as the search direction, while Newton’s method and BFGS use curvature information to construct more efficient directions.

The step size $\alpha^{(t)}$ is determined using line-search conditions. Gradient ascent uses Armijo backtracking to enforce sufficient increase in the ELBO. BFGS uses Wolfe conditions, combining sufficient increase with a curvature condition. The curvature condition is important because it helps ensure

$$\mathbf{y}_t^T \mathbf{s}_t > 0, \quad (2)$$

where $\mathbf{s}_t$ is the parameter-change vector and $\mathbf{y}_t$ is the gradient-difference vector. This positivity condition is required for the BFGS update to preserve positive definiteness of the approximate inverse Hessian.

Exit conditions are applied uniformly across the optimisation methods. Iteration terminates when any of the following conditions is satisfied:

- (i) Gradient norm: $\|\nabla \text{ELBO}\| < \varepsilon_g$.
- (ii) Relative ELBO change: $|\Delta \text{ELBO}|/|\text{ELBO}| < \varepsilon_r$.
- (iii) Parameter change: $\|\Delta \phi\| < \varepsilon_\phi$.
- (iv) Maximum iterations: $t \geq t_{\max}$.

CAVI additionally monitors the maximum relative change in its variational parameters:

$$\max_i \frac{|\phi_i^{(t+1)} - \phi_i^{(t)}|}{|\phi_i^{(t+1)}|} < \varepsilon. \quad (3)$$

The numerical study uses two test cases. The first is a linear Bayesian regression problem, which provides a baseline setting where the assumed regression model is correctly specified. The second is a quadratic Bayesian regression problem, where the design matrix is expanded to include a nonlinear feature x^2 while remaining linear in the coefficient vector. These two cases allow the optimisation methods to be compared under both a simple linear relationship and a more demanding feature-expanded regression problem.

For both test cases, the report compares convergence speed, final ELBO value, computational cost, posterior mean accuracy, posterior variance behaviour, and stability of the covariance estimate. Code for generating the test cases and applying the optimisation methods is included in the report, with the full implementation provided in the appendix.

Optimisation Problem Formulation

Having established that VI converts posterior inference into optimisation, the problem must be stated precisely before any method can be applied. This section identifies the decision variables, the objective function, and the constraints, following the standard form used throughout the course. A well-posed formulation is necessary both for implementation and for meaningful comparison across methods: gradient ascent, Newton's method, and BFGS all operate on the same objective and respect the same constraints, so differences in performance reflect the methods themselves rather than problem differences.

Decision Variables

For Bayesian linear regression with p predictors, the variational parameters (decision variables) are:

- $\boldsymbol{\mu}_\beta \in \mathbb{R}^p$: posterior mean of regression coefficients
- $\boldsymbol{\Sigma}_\beta \in \mathbb{R}^{p \times p}$: posterior covariance (symmetric positive definite)³
- $a_e, b_e > 0$: shape and rate parameters of the variational Gamma factor for precision

Here, a_e and b_e parameterise $q(\tau_e) = \text{Gamma}(a_e, b_e)$; the model prior on τ_e is specified in the Test Case section.

Total dimension: $d = p + \frac{p(p+1)}{2} + 2 = 11$ variables for $p = 3$ predictors.

Objective Function

Maximise the Evidence Lower Bound (ELBO):

$$\text{ELBO}(\boldsymbol{\phi}) = \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] + H(q) \quad (4)$$

$$= \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] - \mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)]. \quad (5)$$

where $\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$ are the variational parameters.

Constraints

- $\boldsymbol{\Sigma}_\beta \succ 0$: Covariance matrix must be positive definite (see also: positive semidefinite, $\mathbf{A} \succeq 0$)⁴
- $a_e, b_e > 0$: Gamma parameters must be positive
- Optional: Minimum variance constraints $\sigma_{\beta_j}^2 \geq \sigma_{\min}^2$ to prevent underdispersion

³A matrix $\mathbf{A}$ is **positive definite** if $\mathbf{v}^T \mathbf{A} \mathbf{v} > 0$ for every non-zero vector $\mathbf{v}$. This means that the matrix amplifies every direction — there is no direction in which it squashes things to zero or flips them negative. For a covariance matrix this means every direction in parameter space has genuine positive uncertainty. Written $\mathbf{A} \succ 0$.

⁴**Positive semidefinite** ($\mathbf{A} \succeq 0$) allows $\mathbf{v}^T \mathbf{A} \mathbf{v} = 0$ for some non-zero $\mathbf{v}$, meaning some combination of coefficients has *exactly* zero uncertainty — the distribution is flat in that direction. A valid covariance matrix only needs to be positive semidefinite, but a semidefinite (not strictly definite) matrix is non-invertible (a matrix is **invertible** if $\mathbf{A}^{-1}$ exists, meaning the equation $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a unique solution for any $\mathbf{b}$; equivalently, none of the eigenvalues are zero). The precision matrix $\boldsymbol{\Sigma}_\beta^{-1}$ appears in the ELBO and CAVI updates, so a non-invertible covariance causes those computations to fail. Such a semidefinite matrix also leads to *degenerate directions*: a **degenerate direction** is one in which the distribution has zero spread (the probability ellipsoid is flat along that direction), which corresponds to a zero eigenvalue and makes the covariance singular.

Constraint Handling via Reparameterisation:

To enforce $\Sigma_\beta \succ 0$, use Cholesky decomposition:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (6)$$

where $\mathbf{L} \in \mathbb{R}^{p \times p}$ is lower triangular with positive diagonal entries. The unconstrained parameters are:

- $L_{ii} = e^{\ell_{ii}}$ for $i = 1, \dots, p$ (exponentiate to ensure positivity)
- $L_{ij} = \ell_{ij}$ for $i > j$ (off-diagonal entries unconstrained)

Total Cholesky parameters: $\frac{p(p+1)}{2}$ elements of ℓ .

To enforce $a_e, b_e > 0$, use log-space parameterisation:

$$a_e = e^{\eta_a}, \quad b_e = e^{\eta_b} \quad (7)$$

where $\eta_a, \eta_b \in \mathbb{R}$ are unconstrained.

Unconstrained parameter vector: $\xi = (\mu_\beta, \ell, \eta_a, \eta_b) \in \mathbb{R}^d$ with $d = p + \frac{p(p+1)}{2} + 2 = 11$ for $p = 3$.

Computational Characteristics

- Gradient computation: $O(np^2)$ per evaluation (dominated by matrix operations)
- Hessian computation: $O(np^2 + d^3)$ for Newton's method
- Parameters exhibit coupling (changing μ_β affects optimal Σ_β)
- Must handle special functions: digamma $\psi(\cdot)$, trigamma $\psi'(\cdot)$ for Gamma distribution

Model and Variational Approximation

The Bayesian regression model used throughout the report is

$$\mathbf{y} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \tau_e^{-1}\mathbf{I}_n) \quad (\text{likelihood}) \quad (8)$$

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \lambda_\beta^{-1}\mathbf{I}_p) \quad (\text{prior on coefficients}) \quad (9)$$

$$\tau_e \sim \text{Gamma}(\alpha_e, \gamma_e) \quad (\text{prior on precision}). \quad (10)$$

The mean-field variational approximation is

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e), \quad (11)$$

$$q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta), \quad (12)$$

$$q(\tau_e) = \text{Gamma}(a_e, b_e). \quad (13)$$

The variational parameter vector is

$$\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e), \quad (14)$$

with dimension

$$d = p + \frac{p(p+1)}{2} + 2. \quad (15)$$

For the quadratic test case with $p = 3$ regression coefficients, this gives $d = 3 + 6 + 2 = 11$ variational parameters.

Overview of Optimisation Methods

Methods considered

- CAVI: Domain-specific baseline exploiting problem structure
- Gradient Ascent: First-order baseline method using gradient information and line search
- Newton's Method: Second-order method with quadratic local convergence near a well-conditioned optimum
- BFGS: Quasi-Newton method that approximates Hessian information; practical general-purpose method for smooth unconstrained problems

Baseline Method: CAVI⁵

Coordinate Ascent Variational Inference (CAVI) is VI-specific. It updates each variational parameter block sequentially using closed-form solutions:

$$\boldsymbol{\Sigma}_\beta \leftarrow \left(\frac{a_e}{b_e} \mathbf{X}^T \mathbf{X} + \lambda_\beta \mathbf{I}_p \right)^{-1} \quad (16)$$

$$\boldsymbol{\mu}_\beta \leftarrow \frac{a_e}{b_e} \boldsymbol{\Sigma}_\beta \mathbf{X}^T \mathbf{y} \quad (17)$$

$$a_e \leftarrow \alpha_e + \frac{n}{2} \quad (18)$$

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \boldsymbol{\Sigma}_\beta)] \quad (19)$$

Properties: Monotonic ELBO increase⁶, linear convergence, $O(np^2)$ per iteration. Provides analytical benchmark for validation.

⁵Blei, Kucukelbir, and McAuliffe (2017), Variational Inference: A Review for Statisticians (Journal of the American Statistical Association).

⁶Monotonic ELBO increase means the objective is non-decreasing across iterations: $\mathcal{L}^{(t+1)} \geq \mathcal{L}^{(t)}$. In CAVI,

Mapping ENEL445 Methods to VI Problem

1. Parameter Ascent (Course: gradient descent)

- Standard iterative update: $\phi^{(t+1)} = \phi^{(t)} + \alpha_t \nabla \text{ELBO}(\phi^{(t)})$
- Line search from Chapter 3.2 (bisection/Wolfe conditions)
- Stopping criteria: $\|\nabla \text{ELBO}\| < \epsilon$
- Convergence characteristics: linear rate expected for strongly concave ELBO

2. Newton's Method

- Standard Newton update: $\phi^{(t+1)} = \phi^{(t)} + \alpha_t [\nabla^2 \text{ELBO}]^{-1} \nabla \text{ELBO}$
- Requires computing and inverting the Hessian matrix
- Computational cost: $O(d^3)$ per iteration for Hessian inversion
- Convergence characteristics: quadratic convergence near optimum (when Hessian is negative definite)
- Challenge: Hessian may not be negative definite away from the optimum

3. BFGS Quasi-Newton (one of five required methods from Project 2)

- BFGS update formula builds inverse Hessian approximation
- Uses gradient differences: $\mathbf{s}_k = \phi^{(k)} - \phi^{(k-1)}$, $\mathbf{y}_k = \nabla \text{ELBO}(\phi^{(k)}) - \nabla \text{ELBO}(\phi^{(k-1)})$
- Computational cost: $O(d^2)$ per iteration
- Convergence characteristics: superlinear convergence expected

Comparison will reveal whether general optimisation methods can compete with or outperform problem-specific algorithms.

each coordinate update maximises the ELBO with respect to one variational factor while holding the others fixed, so each step cannot decrease the ELBO.

Test Cases and Numerical Implementation

The numerical part of the report uses two synthetic regression test cases. Both are generated from known coefficients so that posterior means, posterior standard deviations, and optimisation behaviour can be compared in a controlled setting. The same VI framework is used in both cases; only the design matrix changes.

Purpose of the Test Cases

The linear test case checks the methods under a simple correctly specified regression relationship. The quadratic test case adds nonlinear structure in the input variable while remaining linear in the regression coefficients after feature expansion. Together, the two examples test whether the optimisation methods remain stable as the design matrix becomes more structured and the posterior geometry becomes more coupled.

Test Case 1: Linear Bayesian Regression

The first test case uses the data-generating model

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad (20)$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_e^2), \quad (21)$$

for $i = 1, \dots, n$. The corresponding design matrix is

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad (22)$$

so that $p = 2$ and $\boldsymbol{\beta} = (\beta_0, \beta_1)^T$.

The following Python code generates the linear test case:

```
def generate_linear_case(n=100, sigma=0.5, seed=1):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x])
    beta_true = np.array([1.0, 2.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true
```

This test case is expected to be the easier of the two experiments. CAVI should provide a strong baseline because the model is conjugate, while BFGS is expected to perform well among the general-purpose optimisation methods.

Test Case 2: Quadratic Bayesian Regression

The second test case uses a quadratic data-generating relationship:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i, \quad (23)$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_e^2). \quad (24)$$

Although the relationship between x_i and y_i is nonlinear, the model remains linear in the coefficient vector $\beta = (\beta_0, \beta_1, \beta_2)^T$. This means the same Bayesian linear regression framework can be used after defining the expanded design matrix

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}. \quad (25)$$

The following Python code generates the quadratic test case:

```
def generate_quadratic_case(n=100, sigma=0.5, seed=2):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x, x**2])
    beta_true = np.array([1.0, 2.0, -1.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true
```

This test case is more demanding because the feature-expanded design matrix can create stronger coupling between coefficients. It is therefore useful for testing whether the optimisation method affects convergence and whether posterior variance collapse remains even when the optimiser reaches the same ELBO optimum.

Method Execution Code

The same driver structure is used for both test cases:

```
def run_test_case(generate_case, method_list):
    X, y, beta_true = generate_case()
    results = {}

    for method_name, method_function in method_list.items():
        fit = method_function(X, y)
        results[method_name] = {
            "elbo": fit.elbo_trace[-1],
            "iterations": fit.iterations,
            "mu_beta": fit.mu_beta,
            "sd_beta": np.sqrt(np.diag(fit.Sigma_beta)),
            "beta_error": fit.mu_beta - beta_true,
        }

    return results
```

The full implementation of the ELBO, CAVI, gradient ascent, Newton's method, BFGS, Gibbs sampler, and plotting routines is included in Appendix [Appendix A](#).

Comparison Criteria

For both test cases, the methods are compared using:

- final ELBO value;
- number of iterations to convergence;

- total runtime;
- posterior mean accuracy relative to the known data-generating coefficients;
- posterior standard deviation relative to the Gibbs sampler;
- numerical stability of the covariance estimate;
- sensitivity to initialisation.

Constraint Handling via Reparameterisation

The variational parameters are not free. Σ_β must remain symmetric positive definite throughout optimisation,⁷ and a_e, b_e must remain strictly positive.⁸ Four strategies exist for handling these constraints in a gradient-based optimiser: ignore them (the algorithm crashes when $\log|\Sigma_\beta|$ becomes undefined); add a penalty term (introduces a tuning parameter and ill-conditioning as $\rho \rightarrow \infty$); project back to the feasible set after each step (costs $O(p^3)$ per iteration and is non-differentiable at the boundary, breaking BFGS); or reparameterise so the constraints are encoded structurally and cannot be violated. This work uses reparameterisation throughout—it requires no tuning parameters, no outer loops, and produces a smooth unconstrained problem fully compatible with gradient ascent, Newton, and BFGS. Two transformations are applied: one for the matrix constraint on Σ_β , and one for the scalar positivity constraints on a_e and b_e .

Cholesky Reparameterisation

Σ_β is the covariance matrix of the variational distribution $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, and must therefore remain symmetric positive definite (SPD) at every iteration. A gradient step has no awareness of this constraint: nothing stops an update from producing a matrix with a negative eigenvalue, after which $\log|\Sigma_\beta|$ and Σ_β^{-1} are undefined and the algorithm crashes.

The transformation is $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular. Optimising over $\mathbf{L}$ instead of Σ_β directly ensures any $\mathbf{L}$ produces a valid symmetric positive semidefinite matrix by construction. However, a residual constraint remains: the diagonal entries ℓ_{ii} must be strictly positive for Σ_β to be positive definite rather than merely positive semidefinite. Three options exist: ignore the diagonal constraint (positivity can still be violated, recovering the original problem); apply a softplus function $\ell_{ii} = \log(1 + e^{u_i})$ (valid but less standard); or apply a log-space transformation $\ell_{ii} = e^{\tilde{\ell}_{ii}}$, keeping $\tilde{\ell}_{ii} \in \mathbb{R}$ unconstrained. The log-space option is used here. The parameter count is unchanged at $p(p+1)/2$ free entries: p log-diagonal entries $\tilde{\ell}_{ii}$ and $p(p-1)/2$ unconstrained off-diagonal entries $\ell_{ij}, i > j$.

Log-Space Reparameterisation

The Gamma shape and rate parameters $a_e, b_e > 0$ carry a scalar positivity constraint. The same reparameterisation principle applies: define $\eta_a = \log a_e$ and $\eta_b = \log b_e$, so that $a_e = e^{\eta_a}$ and $b_e = e^{\eta_b}$. Since the exponential function maps $\mathbb{R}$ onto $(0, \infty)$, the optimiser moves η_a, η_b freely over the real line and the positivity constraint is structurally impossible to violate. This is the scalar analogue of the Cholesky diagonal transformation, and the same principle as substituting $x = e^t$ to eliminate $x > 0$ in single-variable optimisation.

After both transformations, the effective optimisation variables are $(\mu_\beta, \tilde{\ell}_{11}, \ell_{21}, \tilde{\ell}_{22}, \dots, \eta_a, \eta_b)$ —all unconstrained reals. The quantities Σ_β, a_e, b_e are recovered on demand when evaluating the ELBO and its gradients.

⁷ Σ_β is the covariance matrix of $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$. A valid covariance matrix must be symmetric positive definite: $\log|\Sigma_\beta|$ and Σ_β^{-1} both appear in the ELBO and its gradient, so a non-positive-definite Σ_β causes immediate numerical failure.

⁸ a_e and b_e are the shape and rate of $q(\tau_e) = \text{Gamma}(a_e, b_e)$. The Gamma distribution is only defined for positive parameters; negative or zero values make $\log \Gamma(a_e)$ and $\psi(a_e)$ (digamma) undefined.

Numerical Implementation Details

This section provides the numerical details used by the implementation. The transformations, line search, regularisation, stopping conditions, and verification checks are applied consistently to both the linear and quadratic test cases.

1. Cholesky Parameterisation

Forward transformation (constrained to unconstrained):

Given $\Sigma_\beta \succ 0$, compute Cholesky decomposition:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (26)$$

where $\mathbf{L}$ is lower triangular with $L_{ii} > 0$.

Extract unconstrained parameters:

$$\ell_{ii} = \log L_{ii} \quad \text{for } i = 1, \dots, p \quad (27)$$

$$\ell_{ij} = L_{ij} \quad \text{for } i > j \quad (28)$$

Inverse transformation (unconstrained to constrained):

Given unconstrained vector ℓ , reconstruct $\mathbf{L}$:

$$L_{ii} = e^{\ell_{ii}} \quad \text{for } i = 1, \dots, p \quad (29)$$

$$L_{ij} = \ell_{ij} \quad \text{for } i > j \quad (30)$$

$$L_{ij} = 0 \quad \text{for } i < j \quad (31)$$

Then compute:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T \quad (32)$$

Gradient transformation (chain rule):

Let $\mathcal{L}(\xi)$ be the ELBO expressed in unconstrained parameters. By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} \frac{\partial \mathcal{L}}{\partial \Sigma_{\beta,km}} \frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ij}} \quad (33)$$

For diagonal entries ($i = j$):

$$\frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ii}} = \frac{\partial}{\partial \ell_{ii}} \left[\sum_s L_{ks} L_{ms} \right] \quad (34)$$

$$= \frac{\partial L_{ki}}{\partial \ell_{ii}} L_{mi} + L_{ki} \frac{\partial L_{mi}}{\partial \ell_{ii}} \quad (35)$$

Since $L_{ii} = e^{\ell_{ii}}$ and $L_{ij} = \ell_{ij}$ for $i \neq j$:

$$\frac{\partial L_{ki}}{\partial \ell_{ii}} = \begin{cases} e^{\ell_{ii}} = L_{ii} & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases} \quad (36)$$

Therefore:

$$\frac{\partial \Sigma_{\beta,km}}{\partial \ell_{ii}} = \delta_{ki} L_{ii} L_{mi} + \delta_{mi} L_{ki} L_{ii} \quad (37)$$

$$= L_{ii} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (38)$$

For off-diagonal Cholesky entries ($i > j$):

$$\frac{\partial L_{ki}}{\partial \ell_{ij}} = \delta_{ki} \delta_{ji} = \begin{cases} 1 & \text{if } k = i, s = j \\ 0 & \text{otherwise} \end{cases} \quad (39)$$

Thus:

$$\frac{\partial \Sigma_{\beta, km}}{\partial \ell_{ij}} = \delta_{ki} L_{mj} + \delta_{mi} L_{kj} \quad (40)$$

Complete gradient formula:

Let $\mathbf{G} = \nabla_{\Sigma_{\beta}} \mathcal{L}$ be the gradient with respect to the covariance matrix. Then:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ii}} = L_{ii} \sum_{k,m} G_{km} (\delta_{ki} L_{mi} + \delta_{mi} L_{ki}) \quad (41)$$

$$= L_{ii} \left(2 \sum_m G_{im} L_{mi} - G_{ii} L_{ii} \right) \quad (42)$$

$$= 2L_{ii} (\mathbf{GL})_{ii} - L_{ii}^2 G_{ii} \quad (43)$$

$$= L_{ii} (2(\mathbf{GL})_{ii} - L_{ii} G_{ii}) \quad (44)$$

For off-diagonal:

$$\frac{\partial \mathcal{L}}{\partial \ell_{ij}} = \sum_{k,m} G_{km} (\delta_{ki} L_{mj} + \delta_{mi} L_{kj}) \quad (45)$$

$$= \sum_m G_{im} L_{mj} + \sum_k G_{km} L_{kj} \quad (46)$$

$$= (\mathbf{GL})_{ij} + (\mathbf{G}^T \mathbf{L})_{ij} \quad (47)$$

$$= ((\mathbf{G} + \mathbf{G}^T) \mathbf{L})_{ij} \quad (48)$$

Implementation: Compute $\mathbf{G} = \nabla_{\Sigma_{\beta}} \text{ELBO}$, then transform to Cholesky space using the above formulas.

2. Log-Space Parameterisation

Forward transformation:

$$\eta_a = \log a_e \quad (49)$$

$$\eta_b = \log b_e \quad (50)$$

Inverse transformation:

$$a_e = e^{\eta_a} \quad (51)$$

$$b_e = e^{\eta_b} \quad (52)$$

Gradient transformation with Jacobian adjustment:

By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \frac{\partial a_e}{\partial \eta_a} = \frac{\partial \mathcal{L}}{\partial a_e} \cdot e^{\eta_a} = a_e \cdot \frac{\partial \mathcal{L}}{\partial a_e} \quad (53)$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial \eta_b} = b_e \cdot \frac{\partial \mathcal{L}}{\partial b_e} \quad (54)$$

Implementation: Compute gradients $\nabla_{a_e} \text{ELBO}$ and $\nabla_{b_e} \text{ELBO}$, then multiply by a_e and b_e respectively.

3. Initialisation Strategy

Primary initialisation (from OLS):

$$\boldsymbol{\mu}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{OLS estimate}) \quad (55)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{I}_p \quad (\text{moderate initial uncertainty}) \quad (56)$$

$$a_e^{(0)} = \alpha_e + \frac{n}{2} \approx 51 \quad (\text{data-informed shape}) \quad (57)$$

$$b_e^{(0)} = 1.0 \quad (\text{vague initial rate}) \quad (58)$$

Rationale: OLS provides good starting point for $\boldsymbol{\mu}_\beta$; identity covariance avoids overconfidence; a_e reflects data contribution.

Alternative initialisation (zero-centred):

$$\boldsymbol{\mu}_\beta^{(0)} = \mathbf{0}_p \quad (59)$$

$$\boldsymbol{\Sigma}_\beta^{(0)} = (\mathbf{X}^T \mathbf{X})^{-1} \quad (\text{frequentist uncertainty}) \quad (60)$$

$$a_e^{(0)} = \alpha_e + 1 = 2 \quad (61)$$

$$b_e^{(0)} = \gamma_e + 1 = 2 \quad (62)$$

Cholesky initialisation:

Given initial $\boldsymbol{\Sigma}_\beta^{(0)}$, compute Cholesky:

$$\boldsymbol{\Sigma}_\beta^{(0)} = \mathbf{L}^{(0)} (\mathbf{L}^{(0)})^T \quad (63)$$

$$\ell_{ii}^{(0)} = \log L_{ii}^{(0)} \quad (64)$$

$$\ell_{ij}^{(0)} = L_{ij}^{(0)} \quad \text{for } i > j \quad (65)$$

Log-space initialisation:

$$\eta_a^{(0)} = \log a_e^{(0)} \quad (66)$$

$$\eta_b^{(0)} = \log b_e^{(0)} \quad (67)$$

4. Line Search: Armijo Backtracking

Algorithm 1 Armijo Backtracking Line Search

Require: Current point $\xi^{(t)}$, search direction $\mathbf{d}^{(t)}$, gradient $\nabla \mathcal{L}^{(t)}$

Require: Parameters: $c_1 = 10^{-4}$ (sufficient decrease), $\rho = 0.5$ (backtracking factor), $\alpha_{\max} = 1.0$

Ensure: Step size α_t satisfying Armijo condition

```

1:  $\alpha \leftarrow \alpha_{\max}$ 
2:  $\text{max\_iter} \leftarrow 50$ 
3:  $k \leftarrow 0$ 
4: while  $k < \text{max\_iter}$  do
5:   Compute trial point:  $\xi_{\text{trial}} \leftarrow \xi^{(t)} + \alpha \mathbf{d}^{(t)}$ 
6:   Evaluate:  $\mathcal{L}_{\text{trial}} \leftarrow \mathcal{L}(\xi_{\text{trial}})$ 
7:   Compute sufficient decrease threshold:  $\mathcal{L}_{\text{threshold}} \leftarrow \mathcal{L}(\xi^{(t)}) + c_1 \alpha (\nabla \mathcal{L}^{(t)})^T \mathbf{d}^{(t)}$ 
8:   if  $\mathcal{L}_{\text{trial}} \geq \mathcal{L}_{\text{threshold}}$  then
9:     return  $\alpha$  // Armijo condition satisfied
10:  end if
11:   $\alpha \leftarrow \rho \alpha$  // Reduce step size
12:   $k \leftarrow k + 1$ 
13: end while
14: Warning: Line search failed, return  $\alpha = 10^{-8}$  (damped step)
15: return  $\alpha$ 

```

Key parameters:

- $c_1 = 10^{-4}$: Standard value for sufficient decrease (Nocedal & Wright)
- $\rho = 0.5$: Halves step size at each backtrack iteration
- $\alpha_{\max} = 1.0$: Start with full Newton step (for Newton and BFGS)
- For gradient ascent: may use $\alpha_{\max} = 0.1$ initially

5. Newton's Method Regularisation

When Hessian $\mathbf{H} = \nabla^2 \text{ELBO}$ is not negative definite (for maximisation), regularise:

$$\mathbf{H}_{\text{reg}} = \mathbf{H} - \lambda \mathbf{I}_d \quad (68)$$

Regularisation parameter selection:

1. Compute eigenvalue⁹ decomposition: $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$ (expensive, $O(d^3)$)
2. Or use Cholesky attempt: try $\mathbf{H} = \mathbf{R}^T \mathbf{R}$; if fails, Hessian not positive definite
3. Simpler approach (Levenberg damping):

$$\lambda = \begin{cases} 0 & \text{if Cholesky succeeds (Hessian negative definite)} \\ \max(10^{-6}, -2\lambda_{\min}) & \text{otherwise} \end{cases} \quad (69)$$

where $\lambda_{\min}$ is smallest eigenvalue of $\mathbf{H}$.

⁹**Eigenvalues** are numbers $\lambda_1, \lambda_2, \dots$ that describe how a matrix scales space along its natural axes (eigenvectors). For a symmetric matrix like Σ_β : all eigenvalues > 0 means positive definite; all eigenvalues ≥ 0 means positive semidefinite; any eigenvalue < 0 means not a valid covariance matrix. They are the stretching factors of the matrix in each principal direction.

Practical implementation (without eigenvalue computation):

Start with $\lambda = 0$. If Cholesky of $-\mathbf{H}$ fails:

$$\lambda \leftarrow 10^{-4} \quad (70)$$

$$\mathbf{H}_{\text{reg}} \leftarrow \mathbf{H} - \lambda \mathbf{I}_d \quad (71)$$

Repeat with $\lambda \leftarrow 10\lambda$ until Cholesky succeeds or $\lambda > 10^3$ (failure).

Newton search direction:

$$\mathbf{d}^{(t)} = -\mathbf{H}_{\text{reg}}^{-1} \nabla \mathcal{L}^{(t)} \quad (72)$$

For maximisation, this should be an ascent direction: $(\nabla \mathcal{L})^T \mathbf{d} > 0$.

6. Convergence Criteria

Implement multiple stopping conditions (logical OR):

1. **Gradient norm:** $\|\nabla \mathcal{L}^{(t)}\|_2 < \epsilon_g = 10^{-6}$
Indicates first-order optimality (stationary point reached).

2. **Relative ELBO change:**

$$\frac{|\mathcal{L}^{(t)} - \mathcal{L}^{(t-1)}|}{1 + |\mathcal{L}^{(t-1)}|} < \epsilon_{\text{rel}} = 10^{-8} \quad (73)$$

Prevents premature termination when ELBO is large in magnitude.

3. **Parameter change:**

$$\|\boldsymbol{\xi}^{(t)} - \boldsymbol{\xi}^{(t-1)}\|_2 < \epsilon_x = 10^{-6} \quad (74)$$

Catches slow convergence in parameter space.

4. **Maximum iterations:** $t \geq t_{\text{max}} = 1000$

Safety check to prevent infinite loops.

Convergence flag assignment:

- Status 0: Gradient norm criterion (ideal)
- Status 1: Relative ELBO change (acceptable)
- Status 2: Parameter change (acceptable but slower)
- Status 3: Maximum iterations (failure to converge)

7. Numerical Gradient Verification

For debugging, verify analytical gradients using finite differences:

Central difference formula:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} \approx \frac{\mathcal{L}(\boldsymbol{\xi} + h \mathbf{e}_i) - \mathcal{L}(\boldsymbol{\xi} - h \mathbf{e}_i)}{2h} \quad (75)$$

where $\mathbf{e}_i$ is the i -th standard basis vector and h is step size.

Step size selection:

For float64 precision ($\epsilon_{\text{machine}} \approx 2.2 \times 10^{-16}$):

$$h = \sqrt{\epsilon_{\text{machine}}} \max(1, |\xi_i|) \approx 1.5 \times 10^{-8} \max(1, |\xi_i|) \quad (76)$$

Error metric:

$$\text{error}_i = \frac{|g_i^{\text{analytical}} - g_i^{\text{numerical}}|}{\max(|g_i^{\text{analytical}}|, |g_i^{\text{numerical}}|, 10^{-8})} \quad (77)$$

Acceptable if $\text{error}_i < 10^{-5}$ for all i .

8. Failure Mode Handling

Common failure modes and recovery:

1. Matrix not positive definite:

- Detection: Cholesky decomposition fails
- Recovery: Add regularisation $\Sigma_\beta \leftarrow \Sigma_\beta + 10^{-6} \mathbf{I}_p$

2. Line search exhaustion:

- Detection: 50 backtracking iterations without Armijo satisfaction
- Recovery: Accept tiny step $\alpha = 10^{-8}$ and continue; if recurring, terminate

3. ELBO decrease:

- Detection: $\mathcal{L}^{(t)} < \mathcal{L}^{(t-1)}$ after line search
- Recovery: Reduce step size aggressively or revert to previous parameters

4. Parameter divergence:

- Detection: $\|\xi^{(t)}\|_2 > 10^{10}$ or any $|\xi_i| > 10^6$
- Recovery: Reinitialise from OLS estimates

5. NaN/Inf in ELBO or gradient:

- Detection: `np.isnan()` or `np.isinf()` checks
- Recovery: Reduce step size or add numerical safeguards (min/max clipping)

Evaluation Criteria

1. Convergence: All methods reach ELBO optimum within tolerance
2. Correctness:
 - CAVI correctness: Gradient/Newton/BFGS implementation is correct if it reaches the same ELBO optimum / variational fixed point as CAVI (same VI objective, same variational family)
 - MCMC correctness: VI is assessed against MCMC as a posterior-accuracy benchmark (means, intervals, calibration), not expected to match exactly because VI is approximate
3. Performance hierarchy:
 - CAVI likely fastest for this conjugate problem
 - BFGS expected to be competitive general-purpose method
 - Newton may struggle with indefinite Hessian
 - MCMC/Gibbs expected slowest computationally, but best for posterior uncertainty accuracy (gold-standard benchmark, not a direct ELBO-optimiser competitor)
4. Numerical verification:
 - Analytical gradients match finite differences within 10^{-5} relative error
 - All methods reach same ELBO value (within 10^{-4})
 - Posterior means agree with CAVI and MCMC within 2 standard errors

Results and Discussion

The results section should report the same quantities for the linear and quadratic test cases so that the methods can be compared directly. For each method, the main reported quantities are the final ELBO value, number of iterations, runtime, posterior mean estimate, posterior standard deviation estimate, and agreement with the Gibbs-sampling reference.

The expected interpretation is as follows. CAVI should be the strongest baseline for the conjugate Bayesian regression setting because it exploits the analytical structure of the model. Gradient ascent provides a first-order comparison method but may require more iterations. Newton's method should converge rapidly near the optimum, but may require Hessian regularisation if the Hessian is not negative definite during early iterations. BFGS should provide the best practical general-purpose compromise because it avoids full Hessian inversion while still using curvature information through gradient differences.

The posterior variance comparison is especially important. If all optimisers reach essentially the same ELBO but still underestimate posterior standard deviations relative to the Gibbs sampler, then the main source of variance collapse is the variational approximation and KL direction rather than the numerical optimiser. If entropy regularisation increases posterior standard deviations while preserving reasonable posterior means, it provides evidence that objective modification is more effective than optimiser substitution for addressing variance collapse.

Conclusion

This report formulates mean-field VI for Bayesian regression as an optimisation problem and applies CAVI, gradient ascent, Newton's method, and BFGS to the resulting ELBO. The linear and quadratic test cases provide controlled numerical settings for comparing convergence, stability, posterior mean accuracy, and posterior variance behaviour.

The central methodological point is that optimisation method and variational approximation play different roles. Better optimisation can improve convergence speed and numerical reliability, but it does not by itself remove the structural underdispersion caused by mean-field factorisation and the exclusive KL divergence. Cholesky and log-space reparameterisations make the optimisation problem numerically safe, while entropy regularisation provides a principled route for directly counteracting posterior variance collapse.

References

- Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877. <https://doi.org/10.1080/01621459.2017.1285773>
- Wainwright, M. J., & Jordan, M. I. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2), 1–305. <https://www.cs.columbia.edu/~blei/fogm/2020F/readings/WainwrightJordan2008.pdf>

Appendix A Python Code Structure

The following code block records the central structure used to generate both test cases and run the optimisation methods. The full implementation can be extended by replacing the method stubs with the CAVI, gradient ascent, Newton, BFGS, and Gibbs-sampling routines described in the main report.

```
import numpy as np

def generate_linear_case(n=100, sigma=0.5, seed=1):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x])
    beta_true = np.array([1.0, 2.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true

def generate_quadratic_case(n=100, sigma=0.5, seed=2):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x, x**2])
    beta_true = np.array([1.0, 2.0, -1.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true

def run_test_case(generate_case, method_list):
    X, y, beta_true = generate_case()
    results = {}

    for method_name, method_function in method_list.items():
        fit = method_function(X, y)
        results[method_name] = {
            "elbo": fit.elbo_trace[-1],
            "iterations": fit.iterations,
            "mu_beta": fit.mu_beta,
            "sd_beta": np.sqrt(np.diag(fit.Sigma_beta)),
            "beta_error": fit.mu_beta - beta_true,
        }

    return results
```

```

# Example usage once method implementations are defined:
# method_list = {
#     "CAVI": fit_cavi,
#     "Gradient Ascent": fit_gradient_ascent,
#     "Newton": fit_newton,
#     "BFGS": fit_bfgs,
#     "Gibbs": fit_gibbs,
# }
# linear_results = run_test_case(generate_linear_case, method_list)
# quadratic_results = run_test_case(generate_quadratic_case, method_list)

```

Appendix B VB Posterior Variance Collapse

Why Variance Collapse Happens

Variational inference finds $q(\boldsymbol{\theta})$ by minimising the KL divergence from q to the true posterior $p(\boldsymbol{\theta} \mid \mathbf{y})$:

$$\text{KL}[q\|p] = \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid \mathbf{y})} d\boldsymbol{\theta} \quad (78)$$

This is called the *forward* (or *exclusive*) KL divergence. The critical asymmetry: this expression is large when q places mass where p is small, but contributes *zero* when q places no mass where p is large. The optimiser therefore drives q to avoid regions where p is small — but has no incentive whatsoever to cover regions where p is large. The result is a q that locks onto the high-probability core of p and ignores the tails, systematically underestimating posterior variance.

This is sometimes called *zero-forcing* behaviour: q is forced to be zero in low-probability regions of p , but is not forced to cover high-probability regions. The reverse direction $\text{KL}[p\|q]$ — called the *reverse* or *inclusive* KL — would instead force q to cover all regions where p has mass, producing over-dispersed approximations. Neither direction recovers the true posterior exactly; standard VI picks the under-dispersed failure mode.

Mean-field factorisation compounds the problem: by forcing $q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta})q(\tau_e)$, any true posterior correlation between $\boldsymbol{\beta}$ and τ_e is set to zero by assumption. Posterior correlations that would otherwise widen the marginals are simply removed, squeezing the approximation tighter.

Observation

Comparison of the VB posterior against the MCMC (Gibbs sampler) gold standard confirms this: the mean-field VB approximation systematically underestimates posterior variance — a well-known failure mode of mean-field variational inference. This will be demonstrated empirically as the first Python task, by running both methods on the same synthetic dataset and computing the SD ratio (VB σ / Gibbs σ) for each parameter. Ratios below 1.0 confirm collapse; the magnitude of the shortfall motivates the constraint-handling approaches below.

Three Approaches to Prevent Collapse

1. Informative Prior on τ_e

Set non-zero prior hyperparameters (α_e, γ_e) to place a lower bound on the posterior variance. A tight prior on τ_e prevents the precision from being driven too high, which in turn keeps $\boldsymbol{\Sigma}_{\boldsymbol{\beta}}$ from collapsing.

Mechanism: The CAVI update for b_e is:

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \boldsymbol{\Sigma}_\beta)] \quad (79)$$

With $\gamma_e > 0$, b_e has a minimum floor, bounding $\mathbb{E}[\tau_e] = a_e/b_e$ from above.

Limitation: Changes the model; requires prior elicitation.

2. Variance Lower Bound Constraint

After each CAVI update, clip $\boldsymbol{\Sigma}_\beta$ so its diagonal never falls below a minimum threshold $\sigma_{\min}^2$:

$$\Sigma_{\beta,jj} \leftarrow \max(\Sigma_{\beta,jj}, \sigma_{\min}^2) \quad \forall j \quad (80)$$

This was already noted in the Constraints section as an optional minimum variance constraint. In code:

```
Sigma_diag = np.diag(self.Sigma_beta)
np.fill_diagonal(self.Sigma_beta,
    np.maximum(Sigma_diag, sigma_min))
```

Limitation: Ad hoc; does not arise from a principled objective modification.

3. Entropy Regularisation of the ELBO

Add a regularisation term proportional to the entropy $H[q]$ of the variational distribution to the objective:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda \cdot H[q(\beta)] \quad (81)$$

For $q(\beta) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta)$, the Gaussian entropy is:

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \boldsymbol{\Sigma}_\beta) = \frac{p}{2} (1 + \log 2\pi) + \frac{1}{2} \log \det(\boldsymbol{\Sigma}_\beta) \quad (82)$$

The regularised gradient with respect to $\boldsymbol{\Sigma}_\beta$ gains an additional term:

$$\nabla_{\boldsymbol{\Sigma}_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\boldsymbol{\Sigma}_\beta} \mathcal{L} + \frac{\lambda}{2} \boldsymbol{\Sigma}_\beta^{-1} \quad (83)$$

This directly rewards larger $\boldsymbol{\Sigma}_\beta$, counteracting the collapse tendency. The hyperparameter $\lambda > 0$ controls the strength of the correction.

This is the most principled of the three approaches and is directly amenable to the gradient-based optimisation methods (gradient ascent, Newton, BFGS) already implemented for this paper.

Selected Approach for This Paper

Entropy regularisation (Approach 3) is used as the selected objective modification and is compared against the unregularised ELBO. The regularisation parameter λ is treated as a hyperparameter and assessed by comparing posterior interval coverage against the MCMC reference. These results form a secondary comparison alongside the main optimiser comparison.

Appendix C Cholesky Reparameterisation: Structural Constraint Satisfaction

The Problem with Direct Optimisation

The covariance matrix Σ_β must satisfy $\Sigma_\beta \succ 0$ (symmetric positive definite) at every iteration. If the optimiser updates Σ_β directly, a gradient step of the form

$$\Sigma_\beta^{(t+1)} = \Sigma_\beta^{(t)} + \alpha_t \nabla_{\Sigma_\beta} \mathcal{L} \quad (84)$$

can produce a matrix that is *not* positive definite. Standard gradient-based methods operate on unconstrained Euclidean space and have no built-in mechanism to respect the positive-definiteness constraint — the constraint is simply violated as a by-product of the update, leading to numerical failure (negative variances, failed Cholesky in later steps, NaN in the ELBO).

The Cholesky Solution: Structural Guarantee

Cholesky decomposition eliminates the constraint by parameterising Σ_β in terms of a lower-triangular factor $\mathbf{L}$:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T, \quad L_{ii} > 0 \quad \forall i \quad (85)$$

The fundamental insight is that **every** lower-triangular matrix $\mathbf{L}$ with strictly positive diagonal entries produces a valid symmetric positive definite matrix:

- **Symmetry:** $(\mathbf{L}\mathbf{L}^T)^T = (\mathbf{L}^T)^T \mathbf{L}^T = \mathbf{L}\mathbf{L}^T$ — automatically symmetric.
- **Positive definiteness:** For any non-zero $\mathbf{v} \in \mathbb{R}^p$,

$$\mathbf{v}^T \mathbf{L}\mathbf{L}^T \mathbf{v} = \|\mathbf{L}^T \mathbf{v}\|^2 > 0 \quad (86)$$

provided $\mathbf{L}^T$ has full column rank, which holds whenever all $L_{ii} > 0$ (since $\mathbf{L}$ is lower triangular, its rank equals the number of non-zero diagonal entries).

It is therefore *structurally impossible* to produce an invalid Σ_β by updating $\mathbf{L}$, regardless of how large or in what direction the gradient step is taken. The constraint is not enforced by a penalty or projection — it is encoded into the *form* of the parameterisation itself.

Unconstrained Optimisation Over ℓ

The remaining difficulty is that $L_{ii} > 0$ is still an inequality constraint. This is resolved by log-parameterising the diagonal:

$$L_{ii} = e^{\ell_{ii}}, \quad \ell_{ii} \in \mathbb{R} \quad (\text{diagonal}) \quad (87)$$

$$L_{ij} = \ell_{ij}, \quad \ell_{ij} \in \mathbb{R} \quad (\text{lower off-diagonal}) \quad (88)$$

Since $e^{\ell_{ii}} > 0$ for all $\ell_{ii} \in \mathbb{R}$, the optimisation is now over a *fully unconstrained* parameter vector $\ell \in \mathbb{R}^{p(p+1)/2}$. No projection, no clipping, no penalty — any point in $\mathbb{R}^{p(p+1)/2}$ maps to a valid $\mathbf{L}$, and hence a valid $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$.

Analogy with Log-Space Parameterisation

This is the same idea used for the Gamma parameters $a_e, b_e > 0$: instead of constraining $a_e > 0$ directly, we optimise over unconstrained $\eta_a = \log a_e \in \mathbb{R}$ and recover $a_e = e^{\eta_a}$, which is always positive. Cholesky + log-diagonal does the same thing for the cone of symmetric positive definite matrices: it provides a smooth bijection between unconstrained Euclidean space and the constraint set, so standard optimisation machinery applies without modification.